



Trabajo Práctico 2

Dado el TAD para secuencias que se encuentra definido en el archivo `Seq.hs`:

- Implementar en un módulo `ListSeq.hs` una instancia para el tipo listas. La implementación debe ser tan eficiente como sea posible. Puede usar el módulo `Par` para paralelizar operaciones.
- Para la implementación dada en **a**, dar la especificación de costos (trabajo y profundidad) de las funciones `filterS`, `reduceS` y `scanS`. No es necesario demostrarlo.
- El tipo `Arr` de arreglos paralelos está dado en el archivo `Arr.hs`. Tiene las siguientes operaciones:
 - `length :: Arr a → Int`
`length a` devuelve el tamaño de el arreglo `a`.
 - `(!) :: Arr a → Int → a`
`a ! i` es la proyección i -ésima del arreglo `a`.
 - `subArray :: Int → Int → Arr a → Arr a`
`subArray i n a` extrae n elementos del arreglo `a` comenzando por el elemento de la posición i .
 - `tabulate :: (Int → a) → Int → Arr a`
`tabulate f n` construye un arreglo `a` de tamaño n tal que $a ! i \equiv f i$ para todo $0 \leq i < n$.
 - `fromList :: [a] → Arr a`
`fromList xs` construye un arreglo `a` a partir de la lista `xs`.
 - `flatten :: Arr (Arr a) → Arr a`
`flatten a` aplana un arreglo de arreglos.

Las operaciones tienen la siguiente especificación de costos:

Operación	W	S
<code>length p</code> <code>p ! i</code> <code>subArray i n a</code>	$O(1)$	$O(1)$
<code>tabulate f n</code>	$O\left(\sum_{i=0}^n W(f i)\right)$	$O\left(\max_{i=0}^n S(f i)\right)$
<code>fromList xs</code>	$O(xs)$	$O(xs)$
<code>flatten a</code>	$O(a) + \sum_{i=0}^{ a -1} O(a ! i)$	$O(\lg a)$

En un módulo `ArrSeq.hs`, dar una instancia de secuencias para el tipo `Arr` que cumpla con la especificación de costos dada en la Tabla 1.

Para evitar conflictos de nombres importar el módulo `Arr` en forma calificada:

```
import qualified Arr as A
```

Luego se puede acceder a las operaciones como `A.length`, etc. Para tener acceso a la operación `!` directamente (sin escribir `A.!`) importarla de la siguiente manera:

```
import Arr (!)
```

- Muestre que la implementación dada en **c** verifica la especificación de costos para la profundidad de la operación `reduceS`.

Operación	W	S
emptyS singletonS lengthS nthS takeS $s\ n$ dropS $s\ n$ showtS s showlS s	$O(1)$	$O(1)$
append $s\ t$	$O(s + t)$	
fromList xs	$O(xs)$	$O(xs)$
joinS s	$O(s) + \sum_{i=0}^{ s -1} O(s_i)$	$O(\lg s)$
tabulateS $f\ n$	$O\left(\sum_{i=0}^{n-1} W(f\ i)\right)$	$O\left(\max_{i=0}^{n-1} S(f\ i)\right)$
mapS $f\ s$	$O\left(\sum_{i=0}^{ s -1} W(f\ s_i)\right)$	$O\left(\max_{i=0}^{ s -1} S(f\ s_i)\right)$
filterS $f\ s$	$O\left(\sum_{i=0}^{ s -1} W(f\ s_i)\right)$	$O\left(\lg s + \max_{i=0}^{ s -1} S(f\ s_i)\right)$
reduceS $\oplus\ b\ s$	$O\left(s + \sum_{(x \oplus y) \in \mathcal{O}_r(\oplus, b, s)} W(x \oplus y)\right)$	$O\left(\lg s \max_{(x \oplus y) \in \mathcal{O}_r(\oplus, b, s)} S(x \oplus y)\right)$
scanS $\oplus\ b\ s$	$O\left(s + \sum_{(x \oplus y) \in \mathcal{O}_s(\oplus, b, s)} W(x \oplus y)\right)$	$O\left(\lg s \max_{(x \oplus y) \in \mathcal{O}_s(\oplus, b, s)} S(x \oplus y)\right)$

Tabla 1: Costos esperados de la implementación con arreglos

Observaciones

- En ambas implementaciones se evalúa corrección y eficiencia.
- Tener especial cuidado con el orden de reducción de `reduceS` y `scanS`.
- Se proveen los módulos `ListTests.hs` y `ArrTests.hs` con algunos casos de tests para las consignas **a** y **c**, respectivamente. Se sugiere agregar más casos convenientemente y utilizarlos para testear las implementaciones.
Para escribir los casos de test se ha utilizado la librería `HUnit` (<https://hackage.haskell.org/package/HUnit>). Puede instalar `HUnit` con cabal de la siguiente manera:

```
cabal update
cabal install --lib HUnit
```

Para correr los casos de test, simplemente abra un intérprete `GHCi`, cargue el módulo en cuestión y evalúe `main`.

Entrega

El trabajo se deberá realizar en grupo de hasta 3 personas. Se deben entregar únicamente:

- Los archivos `ListSeq.hs` y `ArrSeq.hs` con las implementaciones pedidas en **a** y **c**, respectivamente.
- Un documento en formato PDF con los nombres de los integrantes del grupo y las respuestas a las consignas **b** y **d**.

Un único integrante por grupo debe realizar la entrega por Comunidades.